

CEG 2136: Computer Architecture I

Laboratory 0: Introduction to Quartus II Design Software

1 Objective

This laboratory reviews the Altera Quartus II Design Software and the Altera DE2-115 platform. Students will complete the following tasks:

- Review the basic Altera development environment.
- Design a logic circuit using the Graphic Editor.
- Compile, simulate, and debug a full adder design.

Textbook Reading: Section 1-5 Combinational Circuits

2 Introduction

This laboratory serves as a review for the Quartus II design flow. Students are required to design a full adder circuit to reinforce the Altera design flow.

2.1 Functional Specification

The provided full adder utilizes AND, OR, and XOR gates to execute binary addition. Follow the logic diagram to design your circuit.

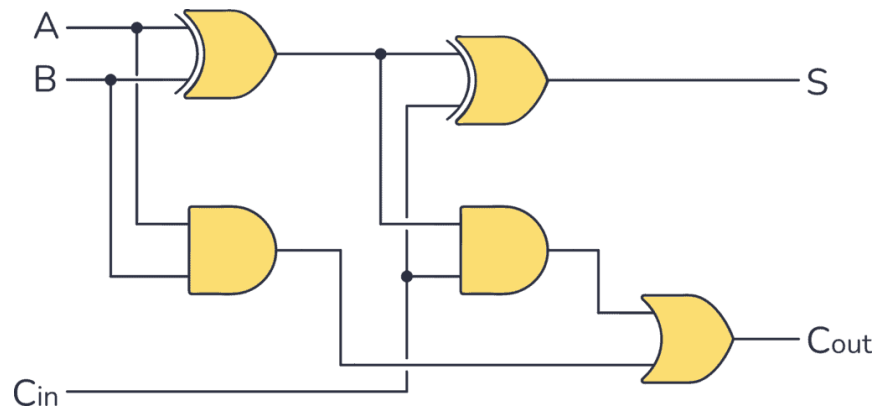


Figure 1: Logic diagram of a full adder $S = A + B + C_{in}$, C_{out} is a carry bit

3 Pre-Lab

Complete the following preparation tasks before beginning the laboratory:

- Read the *General Lab Instructions* on Virtual Campus to understand the course expectations of the laboratory experiments.
- Consult the laboratory calendar to confirm the session duration.
- Review the *Introduction to Altera* on Virtual Campus for a Quartus Design refresher.
- For remote software access, review the *Remote Labs Portal* document and visit [uO-labsPlus](#) for **Omnissa Horizon Client** setup instructions.

4 Laboratory

4.1 Project Setup

1. Launch **Quartus II** and create a new project.
2. Save the project in a dedicated folder and ensure it is backed up to your University OneDrive.
3. Configure the target device as follows:

- Device Family: Cyclone IV E
 - Device: EP4CE115F29C7
4. Ensure the project name and top-level entity are set appropriately.

4.2 Schematic Design

1. Create a new Block Diagram/Schematic (.bdf) file and add it to the project.
2. Implement the full adder using logic gates or symbol blocks, as shown in Figure 1.
3. Add input pins (A , B , C_{in}) and output pins (S , C_{out}).
4. Connect all components according to the full adder logic.
5. Set the schematic file as the top-level entity.
6. Compile the project and verify that it completes with zero errors.

5 Simulation and Verification

1. Create a new University Program waveform file (VWF).
2. Add the input and output signals (A , B , C_{in} , S , C_{out}) to the waveform viewer using the **Node Finder**.
3. Assign clock signals of different frequencies to the input signals to exercise all input combinations. For example:
 - A : 20 ns
 - B : 40 ns
 - C_{in} : 80 ns
4. Verify that the **Quartus II Simulator** is selected under simulation options.
5. Run a functional simulation.
6. Evaluate the output waveform and verify that the results match the expected values in Table 1.

7. Demonstrate the working design to a Teaching Assistant for verification.

Table 1: Truth Table for Full Adder ($S = A + B + C_{in}$)

Inputs			Outputs	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

6 Demonstration

No hardware demonstration is expected for this laboratory.

7 Submission Requirements

No submission is expected for this laboratory.

A Appendix I: Best Practices and Troubleshooting

Use the following references and techniques to guide your laboratory work.

A.1 Project Organization

Always create dedicated folders for each Quartus project. Because laboratory computers erase local files when powered off, you must save and back up your project folders using your University OneDrive to prevent data loss.

A.2 Debugging Connections

To verify signal connectivity in the schematic editor, double-click a wire segment. Quartus highlights the entire connected net, as shown in Figure 2, allowing you to confirm that an input is connected to its intended destination.

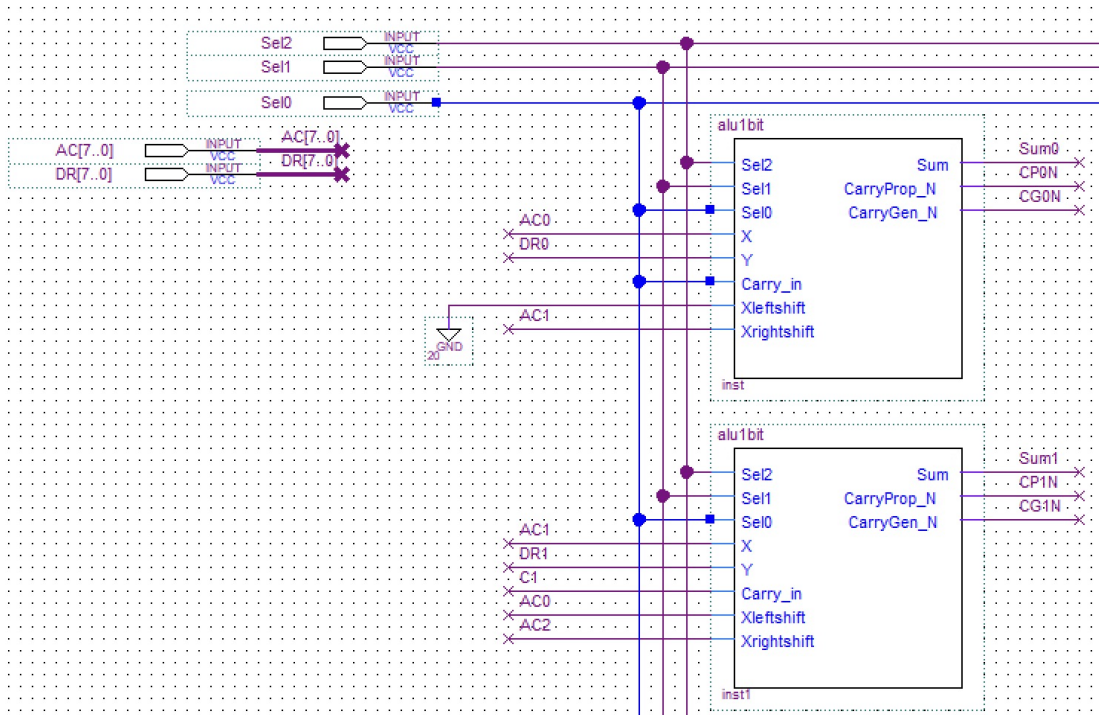


Figure 2: Quartus highlighting a connected net upon double-clicking a wire segment.

A.3 Naming Wires

Name wires to make Quartus recognize connections without drawing the entire physical path [1], as shown in Figure 3.

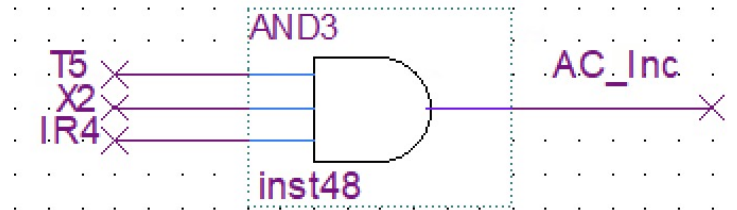


Figure 3: Named wires create logical connections without physical lines.

A.4 Using Bus Connections

Replace multiple individual wires with bus connections to simplify complex schematic designs. Buses reduce visual clutter and lower the probability of wiring errors when routing multiple inputs and outputs. This practice becomes critical as circuit designs grow.

A bus carries multiple data bits. A bus of three signals utilizes the naming format $X[2..0]$ (count down). In Quartus, buses appear as thick wires. You can extract individual signals from buses and use them as regular inputs [3], as shown in Figure 4.

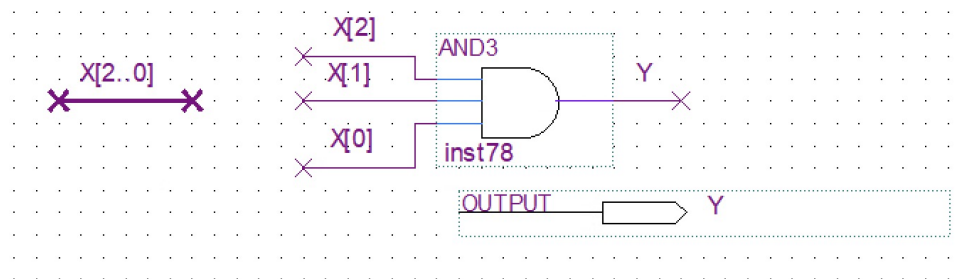


Figure 4: Bus in Quartus, $X[2..0]$, with individual signals used as regular inputs.

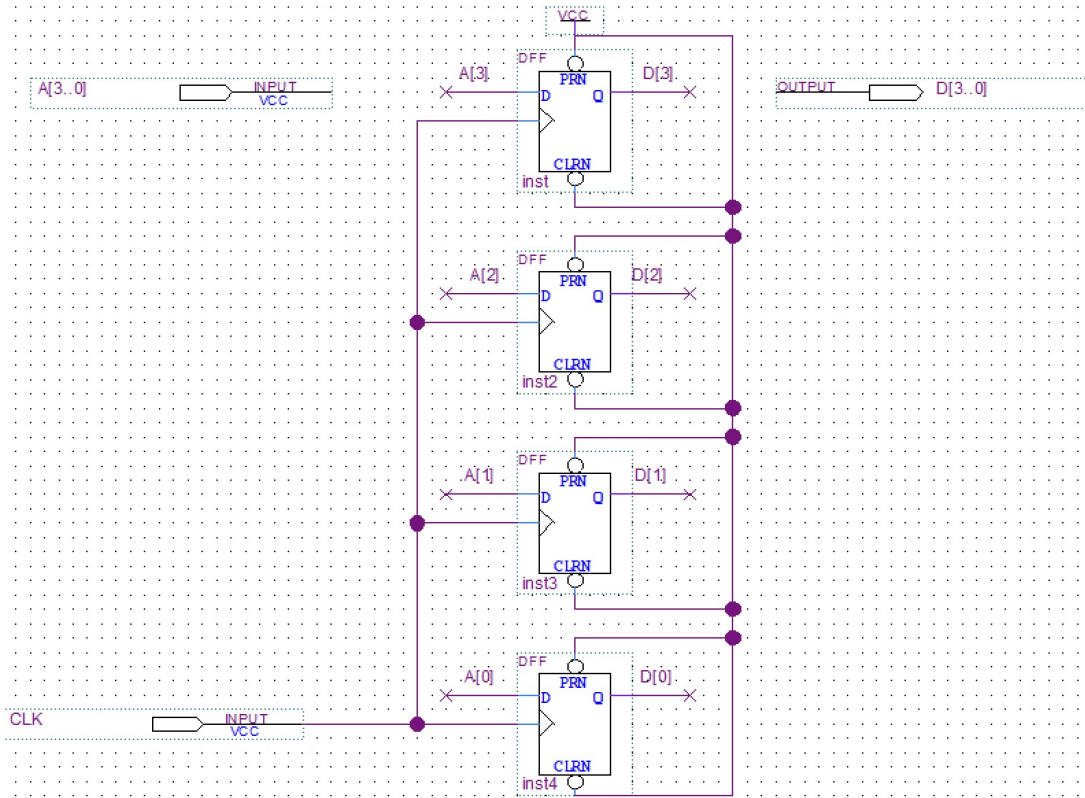


Figure 5: Flip-flop alternate design utilizing bus connections for inputs and outputs.

A.5 Using Wires

In Quartus schematic entry (.bdf) files, you can use a *wire* primitive to assign multiple names to the same signal [3], as illustrated in Figure 6. A *wire* primitive looks like an inverter without the bubble.

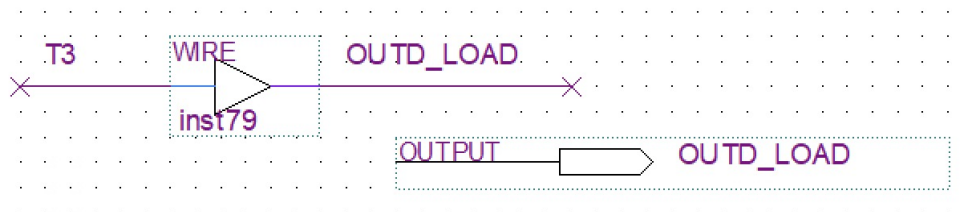


Figure 6: Wire primitive assigning multiple names to a single signal.

A.6 Simplifying Simulation Analysis

In the waveform editor, group related input or output signals together. Select multiple inputs, right-click, choose **Grouping**, and select **Group**.

Once grouped, the radix (base) of the group can be modified. Right-click the group, select **Radix**, and choose binary, hexadecimal, unsigned decimal, or octal. This allows you to input and read values as single numbers (e.g., $A = 1011$), which speeds up verification, as shown in Figure 7. When grouping signals, ensure that the bits are ordered from most significant bit (MSB) to least significant bit (LSB) [1].

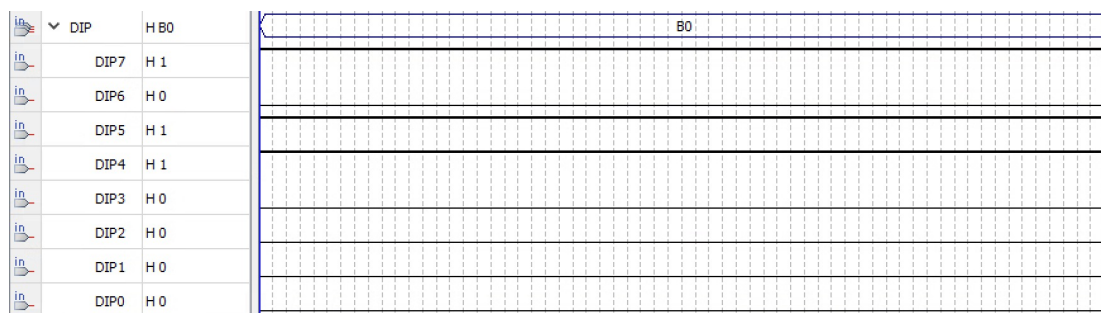


Figure 7: Grouped input signals utilizing a hexadecimal radix.

Note: If bus connections are used in the schematic, Quartus automatically groups these signals in the waveform editor.

A.7 Quartus Software Glitches (Simulation or Compilation)

Occasionally, Quartus workspace files can become corrupted. You might be experiencing this if:

- The simulation window opens and immediately closes without displaying any waveforms.
- The compilation process gets completely stuck, freezes, or runs unusually slowly.

Solution: Create a brand new Quartus project in a new folder. Copy over *only* your essential source files (such as your `.bdf`, `.bsf`, `.v`, `.vhd`, and `.vwf` files) into the new project and recompile.

A.8 Hardware Connection Issues (USB-Blaster)

When attempting to program the FPGA, the USB-Blaster might occasionally fail to appear in the list of available hardware. Try the following:

1. Close and reopen the Hardware Setup window in the Programmer.
2. Unplug and replug the USB cable connecting the board to the PC.
3. Power-cycle the DE2-115 board (turn the red power button off, wait a few seconds, and turn it back on).

Solution: If the USB-Blaster still fails to connect after trying all of the above steps, the specific test board, cable, or PC USB port may be faulty. In this case, try moving to another station and try again.

A.9 Additional Resources

Consult the provided course reference documents for detailed information on Altera Design workflows. The following resources may also be useful for supplementing the Virtual Campus material.

- [Quartus Troubleshooting Guide \(Colorado State University\)](#)
- [Quartus Wire and Bus Connections \(University of Florida\)](#)
- [Quartus Bus Fundamentals \(University of Florida\)](#)
- [Memory Initialization File \(MIF\) Creation in Quartus \(University of Florida\)](#)

B Appendix II: DE2-115 Board I/O Pin Assignments

This appendix provides the FPGA pin assignments for the Altera DE2-115 board, including switches, LEDs, buttons, and 7-segment displays [2].

B.1 Slide Switches and Push-buttons

Table 2: Pin Assignments for Slide Switches

Signal Name	FPGA Pin No.	Description
SW[0]	PIN_AB28	Slide Switch [0]
SW[1]	PIN_AC28	Slide Switch [1]
SW[2]	PIN_AC27	Slide Switch [2]
SW[3]	PIN_AD27	Slide Switch [3]
SW[4]	PIN_AB27	Slide Switch [4]
SW[5]	PIN_AC26	Slide Switch [5]
SW[6]	PIN_AD26	Slide Switch [6]
SW[7]	PIN_AB26	Slide Switch [7]
SW[8]	PIN_AC25	Slide Switch [8]
SW[9]	PIN_AB25	Slide Switch [9]
SW[10]	PIN_AC24	Slide Switch [10]
SW[11]	PIN_AB24	Slide Switch [11]
SW[12]	PIN_AB23	Slide Switch [12]
SW[13]	PIN_AA24	Slide Switch [13]
SW[14]	PIN_AA23	Slide Switch [14]
SW[15]	PIN_AA22	Slide Switch [15]
SW[16]	PIN_Y24	Slide Switch [16]
SW[17]	PIN_Y23	Slide Switch [17]

Table 3: Pin Assignments for Push-button (Debounced) Switches

Signal Name	FPGA Pin No.	Description
KEY[0]	PIN_M23	Push-button [0]
KEY[1]	PIN_M21	Push-button [1]
KEY[2]	PIN_N21	Push-button [2]
KEY[3]	PIN_R24	Push-button [3]

B.2 LED Assignments

Table 4: Pin Assignments for LEDs

Signal Name	FPGA Pin No.	Description
LEDR[0]	PIN_G19	LED Red [0]
LEDR[1]	PIN_F19	LED Red [1]
LEDR[2]	PIN_E19	LED Red [2]
LEDR[3]	PIN_F21	LED Red [3]
LEDR[4]	PIN_F18	LED Red [4]
LEDR[5]	PIN_E18	LED Red [5]
LEDR[6]	PIN_J19	LED Red [6]
LEDR[7]	PIN_H19	LED Red [7]
LEDR[8]	PIN_J17	LED Red [8]
LEDR[9]	PIN_G17	LED Red [9]
LEDR[10]	PIN_J15	LED Red [10]
LEDR[11]	PIN_H16	LED Red [11]
LEDR[12]	PIN_J16	LED Red [12]
LEDR[13]	PIN_H17	LED Red [13]
LEDR[14]	PIN_F15	LED Red [14]
LEDR[15]	PIN_G15	LED Red [15]
LEDR[16]	PIN_G16	LED Red [16]
LEDR[17]	PIN_H15	LED Red [17]
LEDG[0]	PIN_E21	LED Green [0]
LEDG[1]	PIN_E22	LED Green [1]
LEDG[2]	PIN_E25	LED Green [2]
LEDG[3]	PIN_E24	LED Green [3]
LEDG[4]	PIN_H21	LED Green [4]
LEDG[5]	PIN_G20	LED Green [5]
LEDG[6]	PIN_G22	LED Green [6]
LEDG[7]	PIN_G21	LED Green [7]
LEDG[8]	PIN_F17	LED Green [8]

B.3 Clock Inputs

Table 5: Pin Assignments for Clock Inputs

Signal Name	FPGA Pin No.	Description
CLOCK_50	PIN_Y2	50 MHz clock input
CLOCK2_50	PIN_AG14	50 MHz clock input
CLOCK3_50	PIN_AG15	50 MHz clock input
SMA_CLKOUT	PIN_AE23	External (SMA) clock output
SMA_CLKIN	PIN_AH14	External (SMA) clock input

B.4 7-Segment Displays

Table 6: Pin Assignments for 7-Segment Displays (HEX0 to HEX3)

Signal Name	FPGA Pin No.	Description
HEX0[0]	PIN_G18	Seven Segment Digit 0[0]
HEX0[1]	PIN_F22	Seven Segment Digit 0[1]
HEX0[2]	PIN_E17	Seven Segment Digit 0[2]
HEX0[3]	PIN_L26	Seven Segment Digit 0[3]
HEX0[4]	PIN_L25	Seven Segment Digit 0[4]
HEX0[5]	PIN_J22	Seven Segment Digit 0[5]
HEX0[6]	PIN_H22	Seven Segment Digit 0[6]
HEX1[0]	PIN_M24	Seven Segment Digit 1[0]
HEX1[1]	PIN_Y22	Seven Segment Digit 1[1]
HEX1[2]	PIN_W21	Seven Segment Digit 1[2]
HEX1[3]	PIN_W22	Seven Segment Digit 1[3]
HEX1[4]	PIN_W25	Seven Segment Digit 1[4]
HEX1[5]	PIN_U23	Seven Segment Digit 1[5]
HEX1[6]	PIN_U24	Seven Segment Digit 1[6]
HEX2[0]	PIN_AA25	Seven Segment Digit 2[0]
HEX2[1]	PIN_AA26	Seven Segment Digit 2[1]
HEX2[2]	PIN_Y25	Seven Segment Digit 2[2]
HEX2[3]	PIN_W26	Seven Segment Digit 2[3]
HEX2[4]	PIN_Y26	Seven Segment Digit 2[4]
HEX2[5]	PIN_W27	Seven Segment Digit 2[5]
HEX2[6]	PIN_W28	Seven Segment Digit 2[6]
HEX3[0]	PIN_V21	Seven Segment Digit 3[0]
HEX3[1]	PIN_U21	Seven Segment Digit 3[1]
HEX3[2]	PIN_AB20	Seven Segment Digit 3[2]
HEX3[3]	PIN_AA21	Seven Segment Digit 3[3]
HEX3[4]	PIN_AD24	Seven Segment Digit 3[4]
HEX3[5]	PIN_AF23	Seven Segment Digit 3[5]
HEX3[6]	PIN_Y19	Seven Segment Digit 3[6]

Table 7: Pin Assignments for 7-Segment Displays (HEX4 to HEX7)

Signal Name	FPGA Pin No.	Description
HEX4[0]	PIN_AB19	Seven Segment Digit 4[0]
HEX4[1]	PIN_AA19	Seven Segment Digit 4[1]
HEX4[2]	PIN_AG21	Seven Segment Digit 4[2]
HEX4[3]	PIN_AH21	Seven Segment Digit 4[3]
HEX4[4]	PIN_AE19	Seven Segment Digit 4[4]
HEX4[5]	PIN_AF19	Seven Segment Digit 4[5]
HEX4[6]	PIN_AE18	Seven Segment Digit 4[6]
HEX5[0]	PIN_AD18	Seven Segment Digit 5[0]
HEX5[1]	PIN_AC18	Seven Segment Digit 5[1]
HEX5[2]	PIN_AB18	Seven Segment Digit 5[2]
HEX5[3]	PIN_AH19	Seven Segment Digit 5[3]
HEX5[4]	PIN_AG19	Seven Segment Digit 5[4]
HEX5[5]	PIN_AF18	Seven Segment Digit 5[5]
HEX5[6]	PIN_AH18	Seven Segment Digit 5[6]
HEX6[0]	PIN_AA17	Seven Segment Digit 6[0]
HEX6[1]	PIN_AB16	Seven Segment Digit 6[1]
HEX6[2]	PIN_AA16	Seven Segment Digit 6[2]
HEX6[3]	PIN_AB17	Seven Segment Digit 6[3]
HEX6[4]	PIN_AB15	Seven Segment Digit 6[4]
HEX6[5]	PIN_AA15	Seven Segment Digit 6[5]
HEX6[6]	PIN_AC17	Seven Segment Digit 6[6]
HEX7[0]	PIN_AD17	Seven Segment Digit 7[0]
HEX7[1]	PIN_AE17	Seven Segment Digit 7[1]
HEX7[2]	PIN_AG17	Seven Segment Digit 7[2]
HEX7[3]	PIN_AH17	Seven Segment Digit 7[3]
HEX7[4]	PIN_AF17	Seven Segment Digit 7[4]
HEX7[5]	PIN_AG18	Seven Segment Digit 7[5]
HEX7[6]	PIN_AA14	Seven Segment Digit 7[6]

References

- [1] Colorado State University. Quartus troubleshooting guide. <https://www.engr.colostate.edu/ECE102/Spring22/QUARTUS%20TROUBLESHOOTING%20GUIDE.pdf>. Accessed: Feb. 2026.
- [2] Iowa State University. Pin assignment tables. https://home.engineering.iastate.edu/~alexs/classes/2021_Fall_281/labs/Lab_03/DE2-115_PIN_ASSIGNMENTS.pdf. Accessed: Feb. 2026.
- [3] University of Florida. Wire and bus connections in quartus. https://mil.ufl.edu/3701/docs/quartus/wire_bus.pdf. Accessed: Feb. 2026.